

Input con cin

Input con cin

Come leggere dati inseriti dall'utente con cin in C++.

Come si legge l'input dell'utente?

Un programma che fa sempre la stessa cosa con gli stessi dati non è molto utile. `cin` ti permette di leggere quello che l'utente scrive dalla tastiera, rendendo il programma interattivo.

Cos'è `cin` ?

`cin` è lo strumento del C++ per leggere l'input dall'utente tramite tastiera. Come `cout`, fa parte della libreria `<iostream>`.

L'operatore `>>`

L'operatore `>>` "estrae" un valore dall'input e lo salva in una variabile:

```
int x;  
cin >> x;
```

Quando il programma arriva a questa riga, si ferma e aspetta che l'utente scriva qualcosa e prema Invio. Il valore inserito viene salvato nella variabile `x`.

Leggere un numero intero

```
#include <iostream>
using namespace std;

int main() {
    int eta;
    cout << "Inserisci la tua età: "; // spiega cosa vuoi
    cin >> eta;                       // leggi il valore
    cout << "Hai " << eta << " anni." << endl;
    return 0;
}
```

Esempio di esecuzione:

```
Inserisci la tua età: 16
Hai 16 anni.
```

Leggere un numero decimale

```
double altezza;
cout << "Inserisci la tua altezza in metri: ";
cin >> altezza;
cout << "Alta/o " << altezza << " m." << endl;
```

Leggere una parola

Con `cin >>` puoi leggere una parola sola. Il problema: si ferma al primo spazio.

```
string nome;
cout << "Inserisci il tuo nome: ";
cin >> nome;
cout << "Ciao, " << nome << "!" << endl;
```

```
Inserisci il tuo nome: Alice
Ciao, Alice!
```

Se l'utente scrive "Alice Bianchi", `cin >> nome` legge solo "Alice" e scarta "Bianchi".

Leggere una riga intera (con spazi)

Per leggere tutta la riga compreso gli spazi, usa `getline()` :

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string nome_completo;
    cout << "Inserisci il tuo nome completo: ";
    getline(cin, nome_completo); // legge tutta la riga
    cout << "Ciao, " << nome_completo << "!" << endl;
    return 0;
}
```

```
Inserisci il tuo nome completo: Alice Bianchi
Ciao, Alice Bianchi!
```

Leggere più valori

Puoi leggere più valori in una volta sola, separati da spazio:

```
int a, b;
cout << "Inserisci due numeri separati da spazio: ";
cin >> a >> b;
cout << "Somma: " << a + b << endl;
```

Inserisci due numeri separati da spazio: 5 3
Somma: 8

Attenzione: `cin` e `getline` insieme

Se usi `cin >>` e poi `getline()`, c'è una trappola: quando l'utente preme Invio dopo `cin >>`, il carattere "a capo" rimane nel buffer e viene letto subito da `getline()`, saltando l'input dell'utente.

La soluzione è aggiungere `cin.ignore()` per scartare quel carattere "a capo":

```
int eta;
string nome;

cout << "Età: ";
cin >> eta;

cin.ignore(); // scarta il 'Invio' rimasto nel buffer

cout << "Nome completo: ";
getline(cin, nome); // ora funziona correttamente

cout << nome << " ha " << eta << " anni." << endl;
```

Esempio pratico

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string nome;
    int eta;
    double altezza;

    cout << "Come ti chiami? ";
    cin >> nome;

    cout << "Quanti anni hai? ";
    cin >> eta;

    cout << "Quanto sei alto/a (in metri)? ";
    cin >> altezza;

    cout << endl;
    cout << "=== I tuoi dati ===" << endl;
    cout << "Nome:    " << nome << endl;
    cout << "Età:      " << eta << " anni" << endl;
    cout << "Altezza: " << altezza << " m" << endl;

    return 0;
}
```

Esempio di esecuzione:

```
Come ti chiami? Alice
Quanti anni hai? 16
Quanto sei alto/a (in metri)? 1.65

=== I tuoi dati ===
Nome:    Alice
Età:     16 anni
Altezza: 1.65 m
```

Cosa succede se l'utente sbaglia tipo?

Se l'utente scrive del testo dove il programma si aspetta un numero, `cin` entra in uno stato di errore e le letture successive non funzionano. Per ora, puoi supporre che l'utente inserisca sempre il tipo corretto. La gestione degli errori di input è un argomento più avanzato.